



Flexbox, Bootstrap & Responsive Design

Building Layouts That Work Everywhere

Chapter Overview

- 01** Responsive Web Design
- 02** CSS Flexbox Basics
- 03** Flexbox Properties
- 04** Bootstrap Framework
- 05** Putting It Together



Responsive Web Design

Section 1



What is Responsive Web Design?

The Problem

- Users browse on phones, tablets, laptops, and large monitors
- A fixed-width layout that looks great on desktop breaks on mobile

The Solution

- Design web pages that adapt to any screen size automatically
- Content reflows, images resize, navigation restructures
- One codebase serves all devices — no separate mobile site needed



Responsive Design Breakpoints

Responsive Design Breakpoints



Layout Adapts at Each Breakpoint



Use media queries: @media (min-width: 768px) { ... } to change layout at each breakpoint

The Viewport Meta Tag

What It Does

- Tells the browser how to scale the page on mobile devices
- Without it, mobile browsers render at desktop width then shrink to fit

Required in Every Responsive Page

- width=device-width — sets viewport to match the device screen
- initial-scale=1.0 — starts at 100% zoom (no shrinking)
- Place the meta tag inside the <head> element of your HTML



Viewport & Media Queries

```
<!-- Required in every responsive HTML page -->
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">

/* CSS Media Queries – apply styles at specific widths */

/* Mobile-first: base styles for small screens */
.container { padding: 10px; }

/* Tablet and up (768px+) */
@media (min-width: 768px) {
  .container { padding: 20px; max-width: 720px; }
}

/* Desktop and up (1024px+) */
@media (min-width: 1024px) {
  .container { padding: 30px; max-width: 960px; }
}
```



Mobile-First Design

What is Mobile-First?

- Start with styles for the smallest screen, then add complexity
- Use min-width media queries to progressively enhance for larger screens

Why Mobile-First?

- Forces you to prioritize essential content and simplify layout
- Mobile traffic exceeds desktop on most websites today
- Easier to add features for large screens than to remove for small ones



CSS Flexbox Basics

Section 2



What is Flexbox?

- A CSS layout model designed for one-dimensional layouts
- Arranges items in a row or column with powerful alignment controls
- Eliminates the need for floats, inline-block hacks, and complex positioning
- Items can grow, shrink, and reorder dynamically
- Supported in all modern browsers
- Perfect for navigation bars, card layouts, centering, and form alignment



Flex Container & Items

Flexbox: Container & Items

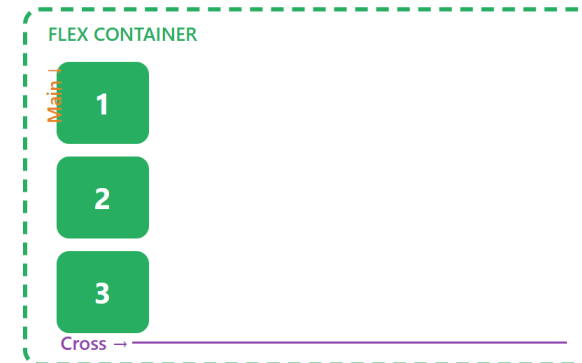
`flex-direction: row` (default)



Container Properties

`justify-content` · `align-items` · `flex-wrap` · `gap`

`flex-direction: column`



Item Properties

`flex-grow` · `flex-shrink` · `flex-basis` · `align-self` · `order`

Getting Started with Flexbox

```
/* Step 1: Create a flex container */
.container {
  display: flex;          /* Activates flexbox */
}

/* All direct children become flex items automatically */

<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>

/* Items line up in a row by default (flex-direction: row) */
/* Items stretch to fill container height by default */
```

Flex Direction & Wrap

flex-direction

- row (default) — items flow left to right
- row-reverse — items flow right to left
- column — items stack top to bottom
- column-reverse — items stack bottom to top

flex-wrap

- nowrap (default) — all items stay on one line, may overflow
- wrap — items wrap to the next line when space runs out



Flex Direction & Wrap Examples

```
/* Vertical stack layout */
.sidebar {
  display: flex;
  flex-direction: column; /* Items stack vertically */
  gap: 10px;              /* Space between items */
}

/* Responsive card grid that wraps */
.card-grid {
  display: flex;
  flex-wrap: wrap;        /* Allow wrapping */
  gap: 20px;
}

.card-grid .card {
  flex: 1 1 250px; /* grow | shrink | min-width */
}
```

Flexbox Properties

Section 3



Flexbox Properties Overview

Flexbox Properties Overview

Container Properties

```
display: flex;  
flex-direction: row | column;  
flex-wrap: nowrap | wrap;  
justify-content: flex-start | ...;  
align-items: stretch | center;  
align-content: flex-start | ...;
```

Item Properties

```
order: 0; (reorder)  
flex-grow: 0; (expand)  
flex-shrink: 1; (shrink)  
flex-basis: auto; (size)  
align-self: auto; (override)
```

justify-content examples:

flex-start



center



space-between



space-around



justify-content (Main Axis)

Controls Horizontal Distribution (in row direction)

- flex-start — items packed to the start (default)
- flex-end — items packed to the end
- center — items centered along the main axis
- space-between — first item at start, last at end, equal gaps between
- space-around — equal padding around each item
- space-evenly — perfectly equal gaps between and around items



align-items & align-self (Cross Axis)

align-items (Container Property)

- stretch (default) — items fill the full cross-axis height
- flex-start — items align to the top of the container
- flex-end — items align to the bottom
- center — items vertically centered

align-self (Item Property)

- Overrides align-items for a single item
- Useful when one item needs different vertical alignment



Centering with Flexbox

```
/* The classic centering problem – solved in 3 lines */  
.centered-container {  
  display: flex;  
  justify-content: center; /* Horizontal center */  
  align-items: center;    /* Vertical center */  
  height: 100vh;         /* Full viewport height */  
}  
  
/* Navigation bar with logo left, links right */  
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 0 20px;  
}
```

flex-grow, flex-shrink & flex-basis

flex-grow

- How much an item grows relative to siblings (0 = don't grow, 1 = grow equally)

flex-shrink

- How much an item shrinks when space is tight (0 = don't shrink, 1 = shrink equally)

flex-basis

- Starting size before growing or shrinking (like width but flex-aware)

Shorthand: flex

- flex: 1 → same as flex: 1 1 0% (grow equally, shrink equally, no base)
- flex: 0 0 200px → fixed 200px, no grow, no shrink



Flex Sizing in Practice

```
/* Sidebar + main content layout */
.layout {
  display: flex;
}

.sidebar {
  flex: 0 0 250px; /* Fixed 250px, never grows or shrinks */
}

.main-content {
  flex: 1;          /* Takes all remaining space */
}

/* Three equal columns */
.equal-columns { display: flex; }
.equal-columns > div { flex: 1; } /* Each gets 1/3 */
```

Bootstrap Framework

Section 4



What is Bootstrap?

Overview

- A free CSS framework for building responsive, mobile-first websites
- Originally created at Twitter — now the most popular CSS framework
- Provides pre-built CSS classes so you write less custom CSS

What You Get

- A responsive 12-column grid system
- Pre-styled components: buttons, cards, navbars, modals, forms
- Utility classes for spacing, colors, typography, and display



Adding Bootstrap to Your Project

```
<!-- Add Bootstrap via CDN in your HTML <head> -->
<link href="https://cdn.jsdelivr.net/npm/
  bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet">

<!-- Optional: Bootstrap JavaScript (for modals, dropdowns) -->
<script src="https://cdn.jsdelivr.net/npm/
  bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>

<!-- That's it! Now use Bootstrap classes in your HTML -->
<button class="btn btn-primary">Click Me</button>
<div class="alert alert-success">It works!</div>
```



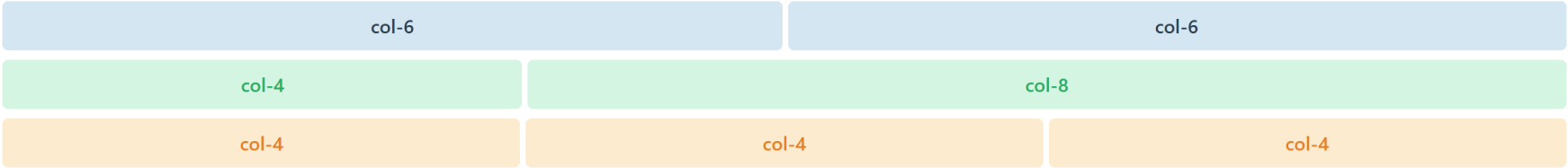
Bootstrap 12-Column Grid

Bootstrap Grid System

12-Column Grid



Layout Examples



Hierarchy



Responsive Prefixes

col-xs-* <576 · col-sm-* ≥576 · col-md-* ≥768
col-lg-* ≥992 · col-xl-* ≥1200 · col-xxl-* ≥1400



Containers, Rows & Columns

Container

- `.container` — fixed max-width that changes at each breakpoint
- `.container-fluid` — always 100% width

Row

- `.row` — a horizontal group of columns (uses flexbox internally)

Column

- `.col` — equal-width column, `.col-6` — half width (6 of 12 columns)
- Responsive: `.col-md-4` → 4 columns on medium screens and up



Bootstrap Grid Examples

```
<!-- Two equal columns -->
<div class="container">
  <div class="row">
    <div class="col">Column 1</div>
    <div class="col">Column 2</div>
  </div>
</div>

<!-- Responsive: stack on mobile, side-by-side on medium+ -->
<div class="container">
  <div class="row">
    <div class="col-12 col-md-8">Main (full → 8/12)</div>
    <div class="col-12 col-md-4">Sidebar (full → 4/12)</div>
  </div>
</div>
```



Bootstrap Breakpoints

Responsive Tiers

- xs (< 576px) — phones in portrait, no class prefix needed
- sm (≥ 576 px) — phones in landscape → .col-sm-*
- md (≥ 768 px) — tablets → .col-md-*
- lg (≥ 992 px) — desktops → .col-lg-*
- xl (≥ 1200 px) — large desktops → .col-xl-*
- xxl (≥ 1400 px) — extra-large screens → .col-xxl-*



Bootstrap Components & Utilities

Commonly Used Components

- Navbar — responsive navigation with hamburger menu on mobile
- Cards — flexible content container with header, body, and footer
- Buttons — `.btn` `.btn-primary`, `.btn-outline-secondary`, `.btn-lg`

Utility Classes

- Spacing: `.mt-3` (margin-top), `.px-4` (padding x-axis), `.mb-0`
- Display: `.d-flex`, `.d-none`, `.d-md-block` (hide on mobile, show on md+)
- Text: `.text-center`, `.text-primary`, `.fw-bold`, `.fs-4`



Putting It Together

Section 5



Flexbox vs. Bootstrap Grid

Use Flexbox When

- You need precise control over alignment and spacing
- Building a single component (navbar, card layout, centering)
- You want to minimize external dependencies

Use Bootstrap Grid When

- You need a full page layout with responsive columns quickly
- Your team benefits from consistent, pre-built patterns
- You also need Bootstrap's components (navbar, modals, forms)



Combining Flexbox & Bootstrap

```
<!-- Bootstrap grid for page layout -->
<div class="container">
  <div class="row">
    <div class="col-md-8">

      <!-- Flexbox for card arrangement inside a column -->
      <div style="display: flex; flex-wrap: wrap; gap: 16px;">
        <div class="card" style="flex: 1 1 200px;">
          <div class="card-body">Card 1</div>
        </div>
        <div class="card" style="flex: 1 1 200px;">
          <div class="card-body">Card 2</div>
        </div>
      </div>

    </div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```



Exercise: Build a Responsive Page

Goal

- Create a responsive landing page using flexbox and Bootstrap

Steps

- Create an HTML file with the viewport meta tag and Bootstrap CDN link
- Add a Bootstrap navbar with your name and 3 navigation links
- Create a 3-column card layout using Bootstrap grid (.col-md-4)
- Use flexbox inside each card to vertically center the content
- Add media queries so cards stack vertically below 768px
- Test by resizing your browser window — content should reflow smoothly



Exercise: Flexbox Navigation Bar

Goal

- Build a navigation bar using only flexbox (no Bootstrap)

Steps

- Create a <nav> element with display: flex and justify-content: space-between
- Add a logo on the left and a of links on the right
- Style the with display: flex and gap: 20px (remove list bullets)
- Use align-items: center to vertically align logo and links
- Add a media query to switch to flex-direction: column below 600px



Key Takeaways

Key Takeaways

1. Responsive design makes one codebase work on all screen sizes
2. Flexbox is the modern CSS tool for one-dimensional layouts and alignment
3. justify-content and align-items control spacing along main and cross axes
4. Bootstrap provides a 12-column grid and pre-built responsive components
5. Mobile-first design starts simple and adds complexity for larger screens
6. Combine flexbox for component layout with Bootstrap grid for page structure

